



COS 214 - Practical 4

Developers

1. Nokhanya Foloti-24578071
2. Nare Maela - 24607917
3. Tebello Mohohoma - 25256387

GitHub Repository:

https://github.com/Mat6577/COS214_Practical4

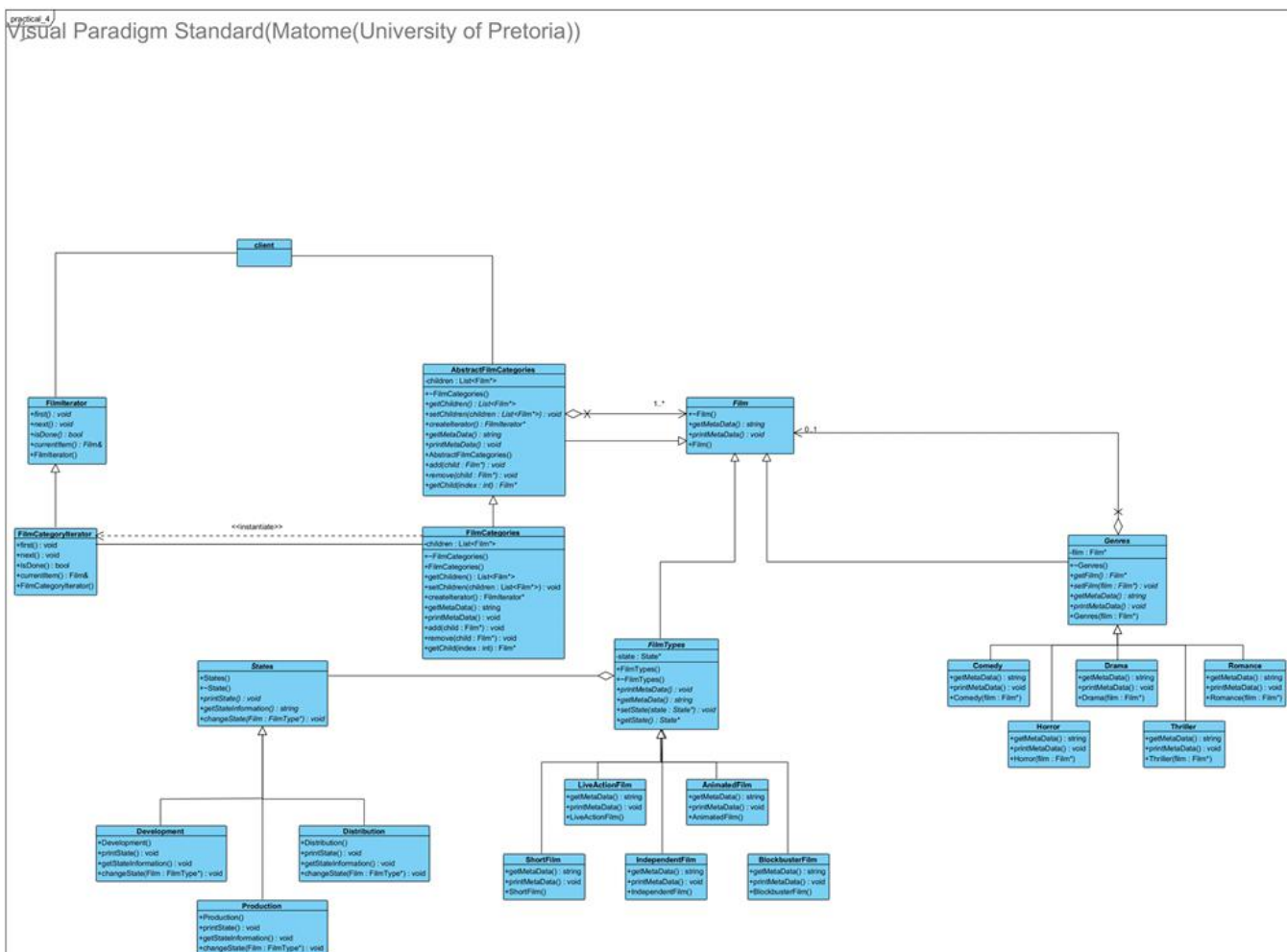
Task 1: Design the System

Short rationale

- The Concrete-Aggregate is supposed to Instantiate Concrete-Iterators. It was difficult to pinpoint which Concrete-Aggregate must instantiate a Concrete-Iterator. In the end we decided to only allow the Concrete-Composite to instantiate the Aggregate. This decision comes from the fact that the composite is the only instance which must traverse the children it contains.
- Adding to the previous point. The decorator holds a single pointer, and this can be easily traversed without an iterator, even when they are stacked on top of each other.

Ownership decisions

- We wanted individual films to have their own states, rather than having all instances inheriting from Film to have a state. That is, it made no sense for a composite and an Abstract decorator to have state.
- What we did was add another abstract layer above the leaves. This abstract layer is what is used to inherit from the Film (Component), then the leaves would inherit from it. This allowed us to add a state pattern which would specifically provide states for the leaves.
- The Film-Category (Concrete-Aggregate) is what instantiates a concrete iterator, but the iterator is responsible for its own memory management.



Traversal-Modification Policy

The system provides two traversal behaviours:

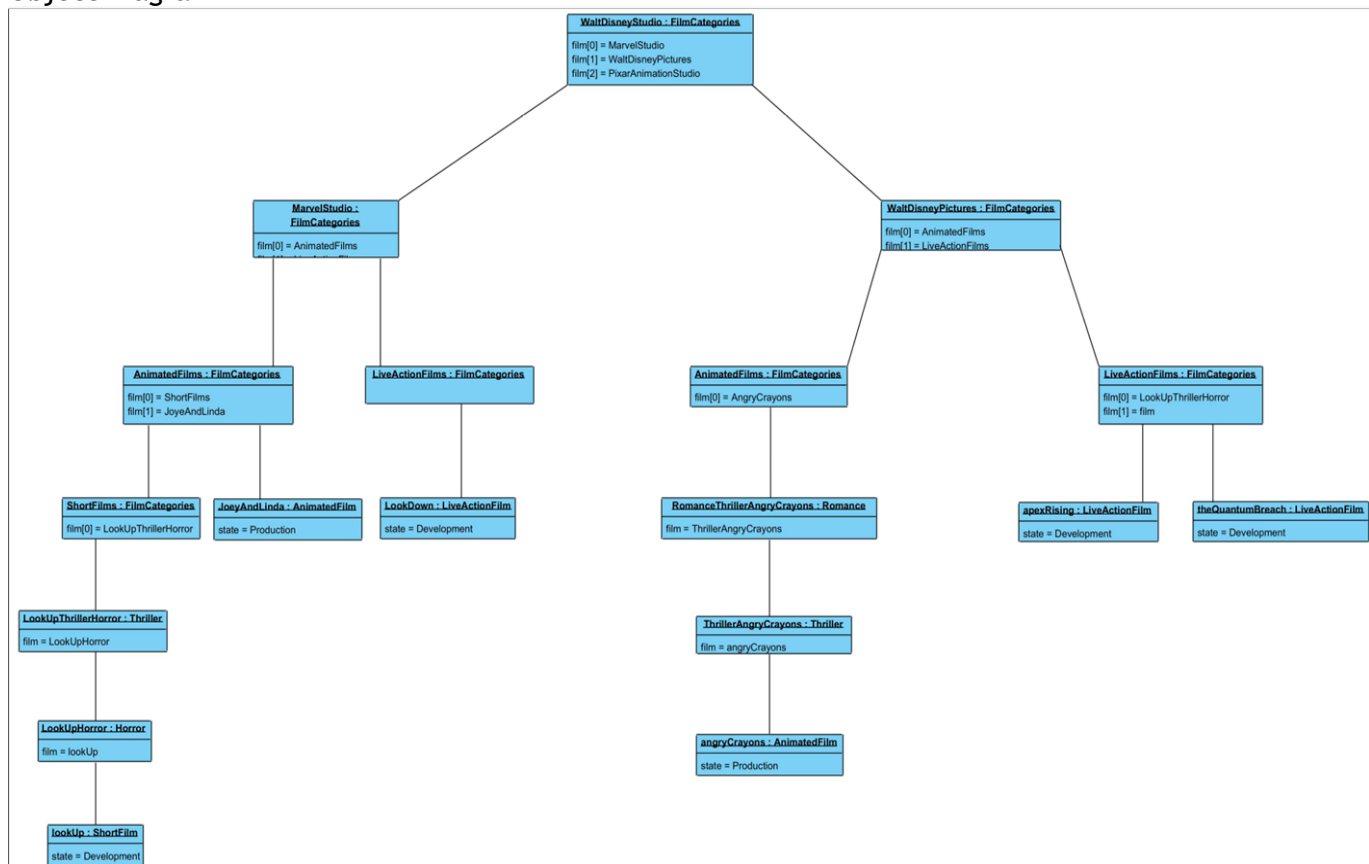
- **FilmCategoryIterator** traverses all films stored in a category.
- **StateBasedIterator** traverses only films whose current state matches the specified state.

When the hierarchy is modified, newly created iterators reflect the updated structure. For example, when an AnimatedFilm is moved from the **Comedy Collection** to the **Featured Collection**, future traversals visit the film in its new category and no longer in its previous category.

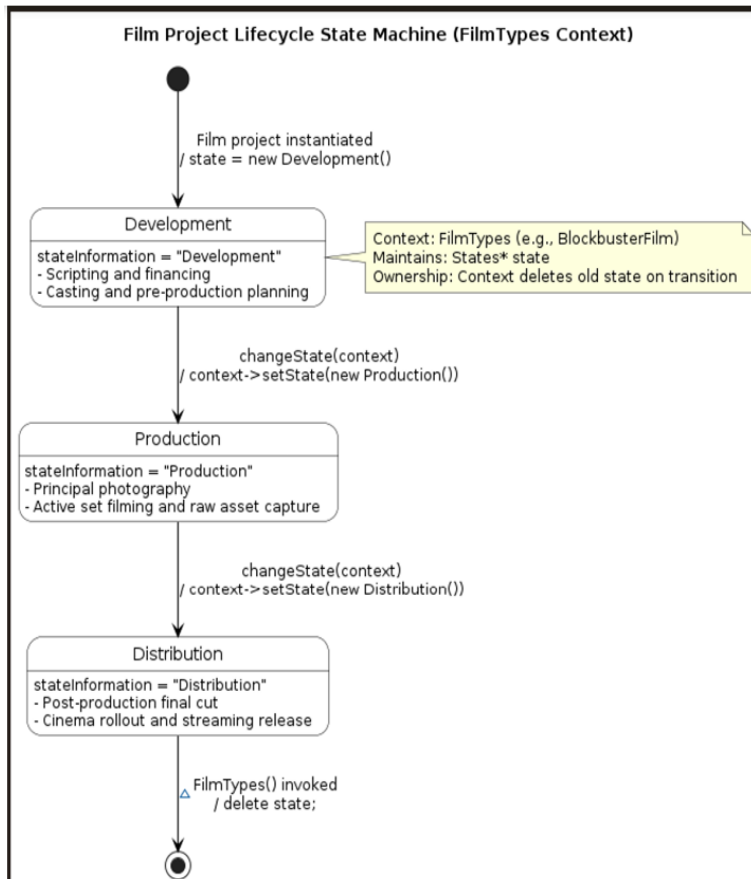
This policy ensures traversal remains consistent with the current structure of the Composite hierarchy.

Task 4: UML Diagram Portfolio

- Object Diagram

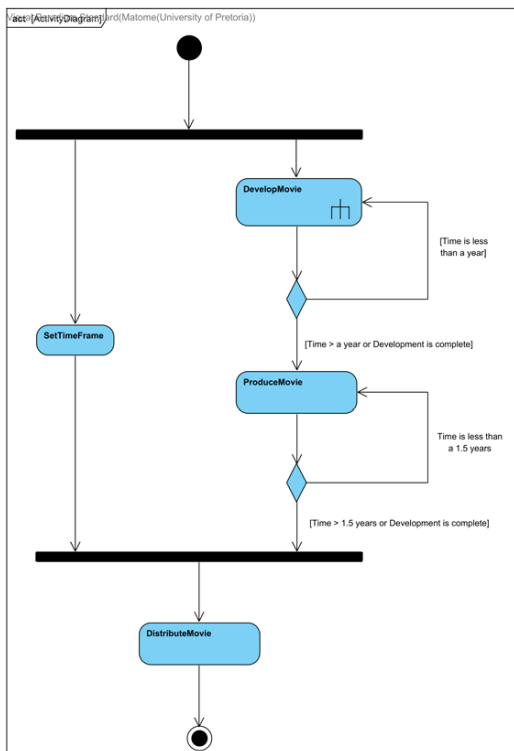


- State diagram

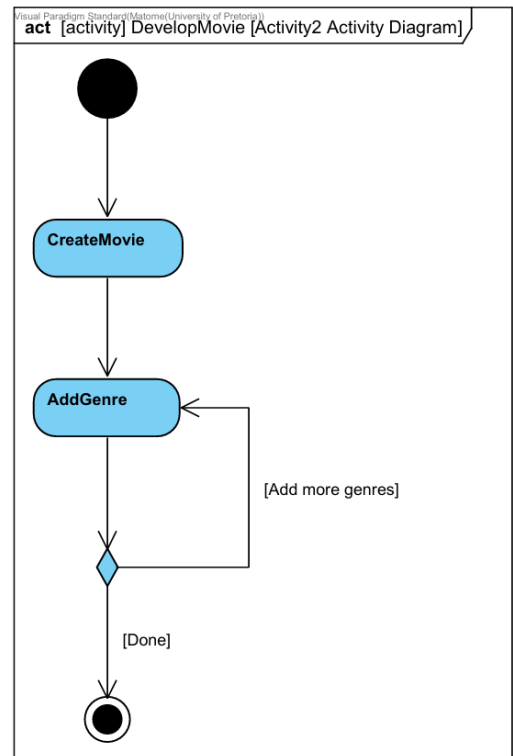


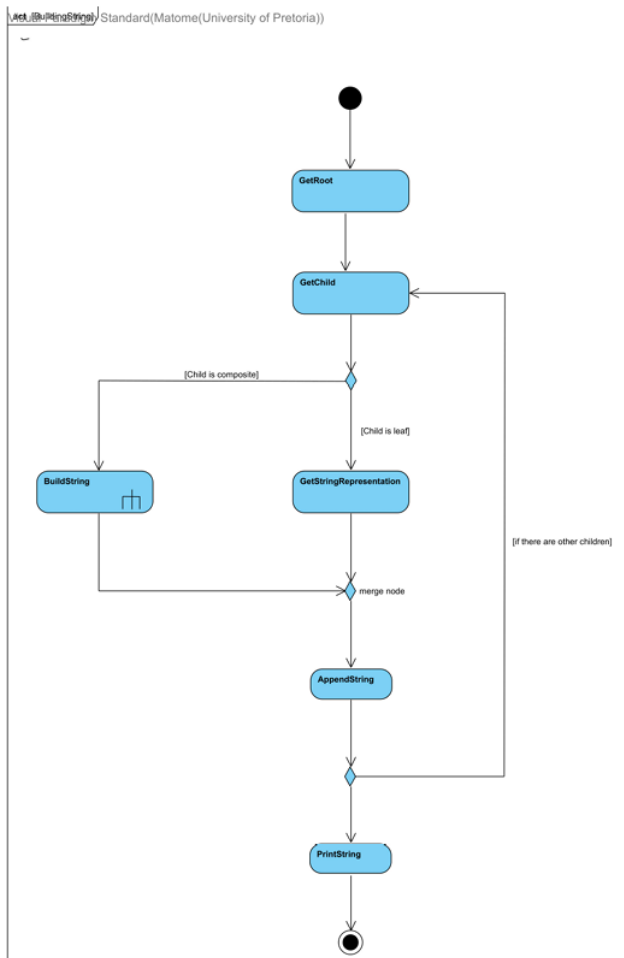
- 3 Activity Diagrams

1.

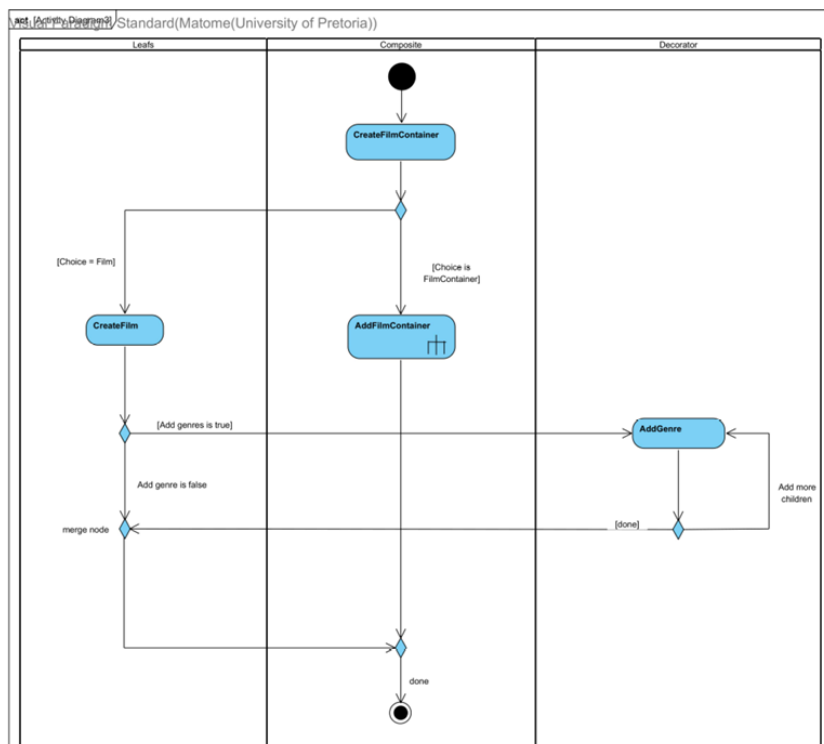


2.





3.



Task 5: Debugging and Memory Investigation

5.1 The application terminated immediately during startup.

Symptom

Running the program produced:
terminate called after throwing an instance of 'char const*'

Aborted

Investigation

The executable was compiled with debug symbols:

```
g++ -g -std=c++11 *.cpp -o taskforge
```

GDB was started:

```
gdb ./taskforge
```

The program was executed:

```
run
```

After the crash, a backtrace was generated:

```
bt
```

Evidence

The backtrace pointed to:

```
Film::Film() at *Film.cpp:10
```

Root Cause

The constructor contained:

```
Film::Film()  
{  
    throw "Not yet implemented";  
}
```

Every Film-derived object triggered this exception during creation.

Correction

The constructor was changed to:

```
Film::Film()  
{  
}  
}
```

Result

The application started successfully and all objects could be created correctly.

5.2 After implementing state transitions, Valgrind reported a memory leak.

Symptom

Valgrind reported:
8 bytes in 1 blocks are definitely lost

Investigation

Used:

```
valgrind --leak-check=full --show-leak-kinds=all ./taskforge
```

Valgrind traced the leak back to state allocation in FilmTypes. The issue occurred during state transitions.

Cause:

We were reassigning a state then losing access to the previous one causing memory leak:

```
void FilmTypes::setState(States* state)
{
    this->state = state;
}
```

Correction

Updated ownership handling so the previous state is released before assigning the new state.

Result

The memory leak was removed.

5.3 After implementing state transitions, Valgrind reported a memory leak.

Symptom

Program finished execution but crashed during shutdown: Segmentation Fault

Investigation

Using gdb:

```
gdb ./taskforge

run

bt
```

Backtrace:

```
Genres::~~Genres()
```

Cause:

```
Genres::~~Genres()
{
    delete film;
}
```

Double deletion the film categories already owns the films so it was causing a double

deletion when genre wanted to delete the film it had already been delete by filmcategory destructor.

Correction

Clarified the ownership and removed the destructor genre just keeps a reference to the films

Result

Seg fault disappeared.

EVIDENCE

```
--- Output/messages ---
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
===== TASKFORGE TEST =====
terminate called after throwing an instance of 'char const*'
Program received signal SIGABRT, Aborted.

--- Output/messages ---
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
===== TASKFORGE TEST =====
terminate called after throwing an instance of 'char const*'
Program received signal SIGABRT, Aborted.
__pthread_kill_implementation (threadid=<optimized out>, signo=6, no_tid=0) at ./nptl/pthread_kill.c:44
warning: 44 ./nptl/pthread_kill.c: No such file or directory
--- Assembly ---
0x00007ffff78a641e  _pthread_kill_implementation+220  mov     %r12d,%edx
0x00007ffff78a6421  _pthread_kill_implementation+223  mov     %ebx,%esi
0x00007ffff78a6423  _pthread_kill_implementation+225  mov     %eax,%edi
0x00007ffff78a6425  _pthread_kill_implementation+227  mov     $0xea,%eax
0x00007ffff78a642a  _pthread_kill_implementation+232  syscall
0x00007ffff78a642c  _pthread_kill_implementation+234  mov     %eax,%r8d
0x00007ffff78a642f  _pthread_kill_implementation+237  neg     %r8d
0x00007ffff78a6432  _pthread_kill_implementation+240  cmp     $0xffffffff,%eax
0x00007ffff78a6437  _pthread_kill_implementation+245  mov     $0x0,%eax
0x00007ffff78a643c  _pthread_kill_implementation+250  cmovbe %eax,%r8d
--- Breakpoints ---
--- Expressions ---
--- History ---
--- Memory ---
--- Registers ---
rax 0x0000000000000000  rbx 0x00000000000000a62  rcx 0x00007ffff78a642c  rdx 0x0000000000000006  rsi 0x00000000000000a62
rdi 0x00000000000000a62  rbp 0x00007ffff78a64b80  rsp 0x00007ffff78a64b40  r8 0x0000000000000016  r9 0x00007ffff78a64291
r10 0x0000000000000000  r11 0x00000000000000246  r12 0x0000000000000006  r13 0x00005555555577530  r14 0x00007ffff78a64d00
r15 0x000055555555635a0  rip 0x00007ffff78a642c  eflags [ PF ZF IF ]  cs 0x000000033  ss 0x00000002b
ds 0x000000000  es 0x000000000  fs 0x000000000  gs 0x000000000  fs_base 0x00007ffff78a6780
--- Source ---
Cannot display "pthread_kill.c"
--- Stack ---
[0] from 0x00007ffff78a642c in __pthread_kill_implementation+234 at ./nptl/pthread_kill.c:44
[1] from 0x00007ffff78a642c in __pthread_kill_internal+234 at ./nptl/pthread_kill.c:89
```

```
Cannot display "pthread_kill.c"
Stack
[0] from 0x00007ffff78a642c in __pthread_kill_implementation+234 at ./nptl/pthread_kill.c:44
[1] from 0x00007ffff78a642c in __pthread_kill_internal+234 at ./nptl/pthread_kill.c:89
[2] from 0x00007ffff78a642c in __GI__pthread_kill+284 at ./nptl/pthread_kill.c:100
[3] from 0x00007ffff7845b7e in __GI_raise+30 at ../sysdeps/posix/raise.c:26
[4] from 0x00007ffff78288ec in __GI_abort+39 at ./stdlib/abort.c:77
[5] from 0x00007ffff7cac275
[6] from 0x00007ffff7cc364a
[7] from 0x00007ffff7cab6c6 in std::terminate()
[8] from 0x00007ffff7cc3901 in __cxa_throw
[9] from 0x000055555555b387 in Film::Film()+73 at Film.cpp:10
[+]
Threads
[1] id 2658 name taskforge from 0x00007ffff78a642c in __pthread_kill_implementation+234 at ./nptl/pthread_kill.c:44
Variables
arg threadid = <optimized out>, signo = 6, no_tid = 0
loc tid = <optimized out>, ret = 0, pd = <optimized out>, old_mask = {__val = {[0] = 11}}, ret = <optimized out>

>>> bt
#0 __pthread_kill_implementation (threadid=<optimized out>, signo=6, no_tid=0) at ./nptl/pthread_kill.c:44
#1 __pthread_kill_internal (threadid=<optimized out>, signo=6) at ./nptl/pthread_kill.c:89
#2 __GI__pthread_kill (threadid=<optimized out>, signo=signo@entry=6) at ./nptl/pthread_kill.c:100
#3 0x00007ffff7845b7e in __GI_raise (sig=signo@entry=6) at ../sysdeps/posix/raise.c:26
#4 0x00007ffff78288ec in __GI_abort () at ./stdlib/abort.c:77
#5 0x00007ffff7cac275 in ?? () from /usr/lib/x86_64-linux-gnu/libstdc++.so.6
#6 0x00007ffff7cc364a in ?? () from /usr/lib/x86_64-linux-gnu/libstdc++.so.6
#7 0x00007ffff7cab6c6 in std::terminate() () from /usr/lib/x86_64-linux-gnu/libstdc++.so.6
#8 0x00007ffff7cc3901 in __cxa_throw () from /usr/lib/x86_64-linux-gnu/libstdc++.so.6
#9 0x000055555555b387 in Film::Film (this=0x555555577430) at Film.cpp:10
#10 0x00005555555583e6 in AbstractFilmCategories::AbstractFilmCategories (this=0x555555577430) at AbstractFilmCategories.cpp:3
#11 0x000055555555b3a9 in FilmCategories::FilmCategories (this=0x555555577430, name="Comedy Collection") at FilmCategories.cpp:7
#12 0x0000555555555ddd5 in main () at main.cpp:19
>>>

G main.cpp M X C FilmCategories.h M X G States.cpp X G FilmCategories.cpp M X G Film.cpp X C AbstractFilmCat
G Film.cpp
1 #include "Film.h"
2
3 Film::~Film()
4 {
5 }
6
7 Film::Film()
8 {
9     // TODO - implement Film::Film
10     throw "Not yet implemented";
11 }
12
```

```
AbstractFilmCategories.cpp X C FilmCategories.h M X G States.cpp X G FilmCategories.cpp M X G Film.cpp M X
G Film.cpp
1 #include "Film.h"
2
3 Film::~Film()
4 {
5 }
6
7 Film::Film()
8 {
9 }
10
```

```
nare7@Nare:/mnt/c/Users/nare7/OneDrive/Documentos/COS214/Prac/Prac4/COS214_Practical4$ valgrind --leak-check=full --show-leak-kinds=all ./taskf
orge
==2816== Memcheck, a memory error detector
==2816== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2816== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2816== Command: ./taskforge
==2816==
===== TASKFORGE TEST =====

Category Metadata:
Comedy Collection

Films in Category:
Short Film
Animated Film
==2816==
==2816== HEAP SUMMARY:
==2816==   in use at exit: 0 bytes in 0 blocks
==2816==   total heap usage: 13 allocs, 13 frees, 75,003 bytes allocated
==2816==
==2816== All heap blocks were freed -- no leaks are possible
==2816==
==2816== For lists of detected and suppressed errors, rerun with: -s
==2816== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

17 void FilmTypes::setState(States* state)
18 {
19     this->state = state;
20 }
```

```
Genres::Genres(Film* film) : film(film) {} // Constructor

Genres::~~Genres()
{
    delete film; // Null check for the destructor
}
```

VALGRIND EVIDENCE

```
valgrind --leak-check=full --show-leak-kinds=all ./taskforge
==4343== Memcheck, a memory error detector
==4343== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==4343== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==4343== Command: ./taskforge
==4343==
===== TASKFORGE TEST =====

=== CATEGORY INFO ===
Comedy Collection

=== ALL FILMS ===
Short Film
Animated Film
Blockbuster Film

=== STATE TRANSITION ===
Before: Development
After: Production

=== PRODUCTION FILMS ===
Short Film

=== MOVING FILM ===

Comedy Collection:
Short Film
Blockbuster Film

Featured Collection:
Animated Film

===== END OF DEMO =====
==4343==
==4343== HEAP SUMMARY:
==4343==   in use at exit: 8 bytes in 1 blocks
==4343==   total heap usage: 26 allocs, 25 frees, 75,336 bytes allocated
==4343==
==4343== 8 bytes in 1 blocks are definitely lost in loss record 1 of 1
==4343==   at 0x485CFD3: operator new(unsigned long) (vg_replace_malloc.c:488)
==4343==   by 0x4007EFC: FilmTypes::FilmTypes() (FilmTypes.cpp:8)
==4343==   by 0x4008341: ShortFilm::ShortFilm() (FilmTypes.cpp:61)
==4343==   by 0x4009E76: main (main.cpp:26)
==4343==
==4343== LEAK SUMMARY:
==4343==   definitely lost: 8 bytes in 1 blocks
==4343==   indirectly lost: 0 bytes in 0 blocks
==4343==   possibly lost: 0 bytes in 0 blocks
==4343==   still reachable: 0 bytes in 0 blocks
==4343==   suppressed: 0 bytes in 0 blocks
==4343==
==4343== For lists of detected and suppressed errors, rerun with: -s
==4343== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Valgrind Evidence

```
make valgrind
g++ -std=c++11 -g -Wall main.cpp AbstractFilmCategories.cpp Comedy.cpp Drama.cpp Film.cpp FilmCategories.cpp FilmCategoryIterator.cpp FilmIterator.cpp FilmTypes.cpp Genres.cpp Horror.cpp Romance.cpp
StateBasedIterator.cpp States.cpp Thriller.cpp -o taskforge
valgrind --leak-check=full --show-leak-kinds=all ./taskforge
==6242== Memcheck, a memory error detector
==6242== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
```

```

==6242== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==6242== Command: ./taskforge
==6242==
===== TASKFORGE DEMO =====

=== GENRE INFORMATION ===

Comedy Metadata:
+++++
+ Age_Classification:          12A / PG-13 +
+ Content_Advisories:          Crude Humour +
+                             Moderate Infrequent Profanity +
+ Scare_Factor_Rating:          0 / 10 +
+ sub_Genre:                   Slapstick / Rom-Com / Satire +
+++++

Horror Metadata:
+++++
+ Age_Classification:          18 +
+ Content_Advisories:          Severe Violence +
+                             Strong Language +
+ Scare_Factor_Rating:          12 / 10 +
+ sub_Genre:                   Slasher / Splatter / Gore Fiesta +
+++++

Romance Metadata:
+++++
+ Age_Classification:          PG-13 +
+ Content_Advisories:          Sensual Content +
+                             Romantic Dialogue & Mild Language +
+ Scare_Factor_Rating:          0 / 10 +
+ sub_Genre:                   Contemporary Romance / Historical Drama / Rom-Com +
+++++

=== CATEGORY INFORMATION ===
Comedy Collection

=== ALL FILMS ===
Short Film
Animated Film
Blockbuster Film

=== FILM LIFECYCLE ===

Short Film State:
Current State: Development
After First Transition: Production
After Second Transition: Distribution

=== PRODUCTION FILMS ===
Animated Film

=== MOVING FILM ===

=== COMEDY COLLECTION ===
Short Film
Blockbuster Film

=== FEATURED COLLECTION ===
Animated Film

===== END OF DEMO =====
==6242==
==6242== HEAP SUMMARY:
==6242==    in use at exit: 0 bytes in 0 blocks
==6242==   total heap usage: 40 allocs, 40 frees, 78,964 bytes allocated
==6242==
==6242== All heap blocks were freed -- no leaks are possible
==6242==
==6242== For lists of detected and suppressed errors, rerun with: -s
==6242== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from

```

GitHub workflow reflection

Initially it was difficult to work as a group using Git/GitHub. This is mainly attributed to the fact that were new to the language and platform. However, as we progressed, we managed to have a system going which allowed us to effectively work together and progress with the practical. There were situations where some of us lost our commits due to miscommunication within the team.

How we divided the work:

We assigned a team leader who made the final calls, this helped significantly as they managed the pulls which were pushed by other team members ensuring that the code does not have bug leading to corruption of the main branch.

How changes were integrated:

Each team member created their own branch and worked on their designated section of the code. Then they would push if it was working properly. Once they had completed their tasks, they merged their branch into the main branch. There were issues with the merging, so we had to come up with a solution to tackle that by moving through the different branches to copy the files and paste the right one. Although not the best solution, it was a learning curve on how to account for people working on different branches for future projects. Once the integration issues were resolved, minor changes and fixes were pushed directly to the main branch, while larger features

continued to be developed in separate branches before being merged.

How requests helped:

They ensured that the main branch only contained working code as the pulls were overviewed.

Team contribution statement

The project was completed collaboratively, with each team member contributing according to their strengths. Responsibilities included implementing design patterns, producing UML and design documentation, integrating code, debugging, testing, and managing the GitHub workflow. All team members contributed to the final system and its successful completion.

